neuro-san-esp: a beginner's guide

What it does, how it runs, and what every command prints.

neuro-san is a framework from Cognizant AI Lab for building teams of AI agents that work together. It can design such a team from one sentence. What it cannot do is tell you whether the team it designed is any good, or whether a different team would do the same job better and cheaper.

This project adds that missing half. It **gives every team the same exam**, scores the result, and **searches for a better team** by copying good ones with small changes. A cheap predictor guesses which changes are worth testing, so only the promising ones are paid for.

> **The whole project in one picture**
>
> Invent many agent teams → give each the same exam → score each on correct answers, cost and size → keep the best, change them a little, repeat. A predictor skips testing the changes that look bad.

Words you will meet

Word	What it means here
Agent	One AI model with a job description and, sometimes, a tool.
Agent network	A team of agents. One is the router (the boss); the others are specialists it can ask.
Router	The agent a question arrives at. It decides which specialist to ask next.
Tool	Something an agent can use. Here: a search over the company's documents.
Token	A small piece of text. Models are paid for by the token, so tokens are the cost.
Multi-hop	A question that needs several documents chained together.
Accuracy	Share of exam questions answered correctly.
Fitness	One score from accuracy, tokens and team size. Higher is better.
Genome	The full description of one team, as neuro-san reads it.
Mutation	A small change to a team: add an agent, rewire, change a model.
Predictor	A small machine-learning model that guesses a team's score from its design, without running the exam.
Held-out	Questions kept aside and never used to choose a winner, so the winner can be judged fairly.

The exam

An invented company, so no model can answer from memory.

The exam is about **Meridian Logistics**, a company that does not exist: 24 depots (warehouses), 40 customer contracts and 60 late-delivery incidents, written as 124 short documents. Because it is invented, a model can only answer by searching the documents. Here are three of them, exactly as an agent sees them:

```
Contract C-2100
Client: Verity Health
Serviced by depot: D19
Goods: refrigerated pharmaceuticals
Late-delivery penalty: 250 per hour
Annual value: 2472000

Meridian Logistics depot D19
Location: Pickering
Depot manager: M. Kowalczyk
Loading bays: 16
Opened: 1998

Incident INC-4400 (2025)
Affected contract: C-2100
Hours late: 23
Cause: storm damage
```

One question, answered by hand

Question T02: Which city is the depot that services contract C-2100 located in?

Hop 1. Find contract C-2100. It says it is serviced by depot **D19**.
Hop 2. Find depot D19. It says its location is **Pickering**.
Answer: Pickering. Two documents, so this is a two-hop question.

The question sets

Set	Questions	Used for
The benchmark	17	Every committed measurement: 1 to 4 hops.
The bank	250	Held-out checks: 1 to 9 documents, entities the benchmark never names.
meridian-select	60	What the scale-up search chooses on; 40% whole-company totals.
meridian-judge	100	Only for judging winners; shares no entity with the select set.

Every answer is computed from the same seed that writes the documents, and a test re-derives each one from the document text, so the exam itself cannot be wrong.

How a team answers

What happens between the question and the reply.

Step 1. The question reaches the router

The router reads its instructions and the list of specialists it may ask, each with a one-line description.

Step 2. The router asks a specialist

For the question above it asks the contract specialist about the contract, gets back the depot code, then asks the depot specialist about that depot.

Step 3. Specialists search the documents

Each specialist calls the search tool, which returns the three documents that best match its query, and reports what they say.

Step 4. The router replies

In an exam run it replies with the bare answer so it can be marked; on the web page it explains which documents it used.

Every one of those steps is a paid model call. A two-hop question is typically about ten calls; that is why tokens are the cost.

How a team is scored

Each reply is marked right when it contains the known answer. For a number it must be the one number the reply gives: a reply that lists several numbers is marked wrong even if the right one is among them, and repeating the question earns nothing.

After the exam each team gets one number:

```
fitness = accuracy - 0.06 x min(tokens / 300000, 1) - 0.02 x (agents / 9)
```

Team	Accuracy	Tokens	Agents	Fitness
The designer's team	0.8235	192,640	4	0.7761
The best evolved team	0.9412	179,800	5	0.8941

Worked through for the designer's team: 0.8235 − 0.06 × 0.642 − 0.02 × 4/9 = **0.7761**. Correct answers matter most; among equally correct teams, the cheaper and smaller one wins.

How the search finds a better team

Four phases, repeated.

Step 1. Phase A: test the starting teams

Three hand-made teams, one shaped the way neuro-san's designer builds them, sit the real exam. This costs model calls.

Step 2. Phase B: train the predictor

A small model learns, from the teams already tested, how a team's design relates to its accuracy and its token cost.

Step 3. Phase C: breed many children, for free

The best teams are copied with one small change each (seven kinds of change). Invalid teams are thrown away. The predictor guesses every child's score in a fraction of a millisecond.

Step 4. Phase D: test only the promising few

The children the predictor likes best sit the real exam, and their results go back into Phase B.

Example: the change that won

Both winning teams came from the same change, **reassign_model**: the router was moved to a stronger model while every specialist stayed on the cheap one. A per-agent model is a setting neuro-san already has and nothing tunes; the search found it.

What the free half prints

`make offline` runs phases B and C on the committed measurements with no key and no cost. Captured output:

```
No local measurements in .esp-cache -- falling back to the committed ones in tests/fixtures/cache.
Run `make baseline` with a key to train on your own instead.

Phase B -- training the Predictor on 12 real evaluations from tests/fixtures/cache
  one model per outcome objective; fitness is derived from their predictions, not learned
  outcome surrogate on 12 samples: fitness +0.669 (accuracy +0.650 (null -0.138), tokens -0.501 (null -0.074))
  the null in brackets is what this same cross-validation returns on shuffled targets --
  on a population this small it is not zero, so the margin over it is the real figure:
    accuracy  margin over its own null: +0.787
    tokens    margin over its own null: -0.427
  !! the tokens model does not beat its own null (margin -0.427). It is excluded from the ranking below, as a service

Phase C -- evolving 2000 candidates against it
  348 distinct viable, 7475 rejected as invalid, 62942 duplicates, 9235 already measured
  NOTE: asked for 2000 and the reachable space yielded 348. The numbers below describe what was actually generated.
  bred in 8.34s, scored in 0.02s, zero provider calls
  ~0.066 ms per candidate to rank

  21 candidates tie at the top (+0.8962). The Predictor cannot separate them; among them the order below is generation

Top 5 by predicted fitness:
  +0.8962  f10a8379aed06bc7  agents=5 depth=2  via reassign_model
  +0.8962  a1e6f5fa98bf20a7  agents=5 depth=2  via reassign_model
  +0.8962  f92bf210d0ab1ba4  agents=5 depth=2  via reassign_model
```

Running it yourself

Every command, in the order you would use it.

Step 1. Install (no key needed)

Python 3.12 or newer.

```
python3.12 -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"
```

Step 2. Check everything works

Lint, the docs, neuro-san's own validator, every test, and a real offline search. It should end with the tests passing.

```
make validate
```

Step 3. Explore the committed results, still free

These read the measurements already in the repository.

```
make offline       # breed and rank candidates, zero model calls
make holdout       # choose on half the questions, judge on the other
make ablation      # does the predictor pick better than chance?
make bank-report   # the held-out bank comparison
make experiment    # the scale-up plan and its price; spends nothing
```

Step 4. Add a key for real runs

Copy the example settings file and put one provider's key in it. The file is never committed.

```
cp .env.example .env     # then edit .env: GOOGLE_API_KEY=...
make check-key           # asks the provider if the key works
make smoke               # four questions to the best team
```

Step 5. Talk to the teams in a browser

```
python apps/web/serve.py   # http://localhost:7860
make studio                # neuro-san's own UI, http://localhost:4173
```

The web page has two tabs: **Ask** sends a question to the best team and marks benchmark questions right or wrong in front of you; **Measure** runs up to four teams on the same questions and shows which is best on correctness and cost.

Step 6. Measure your own team

Any neuro-san network, any questions.

```
make measure NETWORK=registries/my_network.hocon TASKS=my_questions.jsonl

# my_questions.jsonl, one question per line:
{"question": "Which city is depot D08 in?", "answer": "Eastgate"}
```

What it has found so far

In plain words, with what it does not show.

Measurement	Result
12 teams on the 17-question benchmark	The best evolved team answered 16 of 17, the designer's team 14 of 17.
24 new questions (the bank)	Designer 23, evolved 21: the advantage did not show on new questions. Too few to prove either way.
20 harder select questions	Designer 16 of 20: hard enough to tell teams apart.
The predictor, offline	Picks the best of three unseen teams 62% of the time, against 33% by chance. Picking the team with the most agents does as well.

What is still open

Whether the predictor makes the search better for the same money. The pool benchmark that answers it (make pool GO=1) is built and rehearsed for $0, and needs a paid key: 9,000 question-runs, about $61 to $125 on Claude Haiku. Even then one pool can only show a large difference; a small one needs more questions or a second pool.

Where things are

Folder	What is inside
esp/eval/	The invented company, the question sets, the exam runner
esp/genome/	Team descriptions and the seven kinds of change
esp/surrogate/	The predictor
esp/evolve/	The search loop and the scale-up experiment
apps/web/	The browser page
tests/	The checks, and the 12 committed measurements
docs/	This guide, the design notes, the findings, the PDFs